

Title: Custom Embedded Linux Distribution for Raspberry Pi 4 using Yocto.

Objective: To build a complete Linux operating system from source code for Raspberrypi-4
-Instead of using preexisting OS like Ubuntu or rasperry pi OS we create our own OS.

Project details:

A major open-source collaboration that provides tools and templates to help developers build custom, lightweight, and secure Linux operating systems tailored to specific hardware architecture.

Yocto is generally a build framework

Linux Kernel + Packages + Configuration



Yocto



Custom Linux Image

Why only Yocto?

Without yocto a custom-made OS is hard to build, but with yocto the build can be completely customizable.

Source Code (RAW code for the OS build)



Recipes (Instruction on how to build / custom changes)



BitBake (Toll/Build toll)



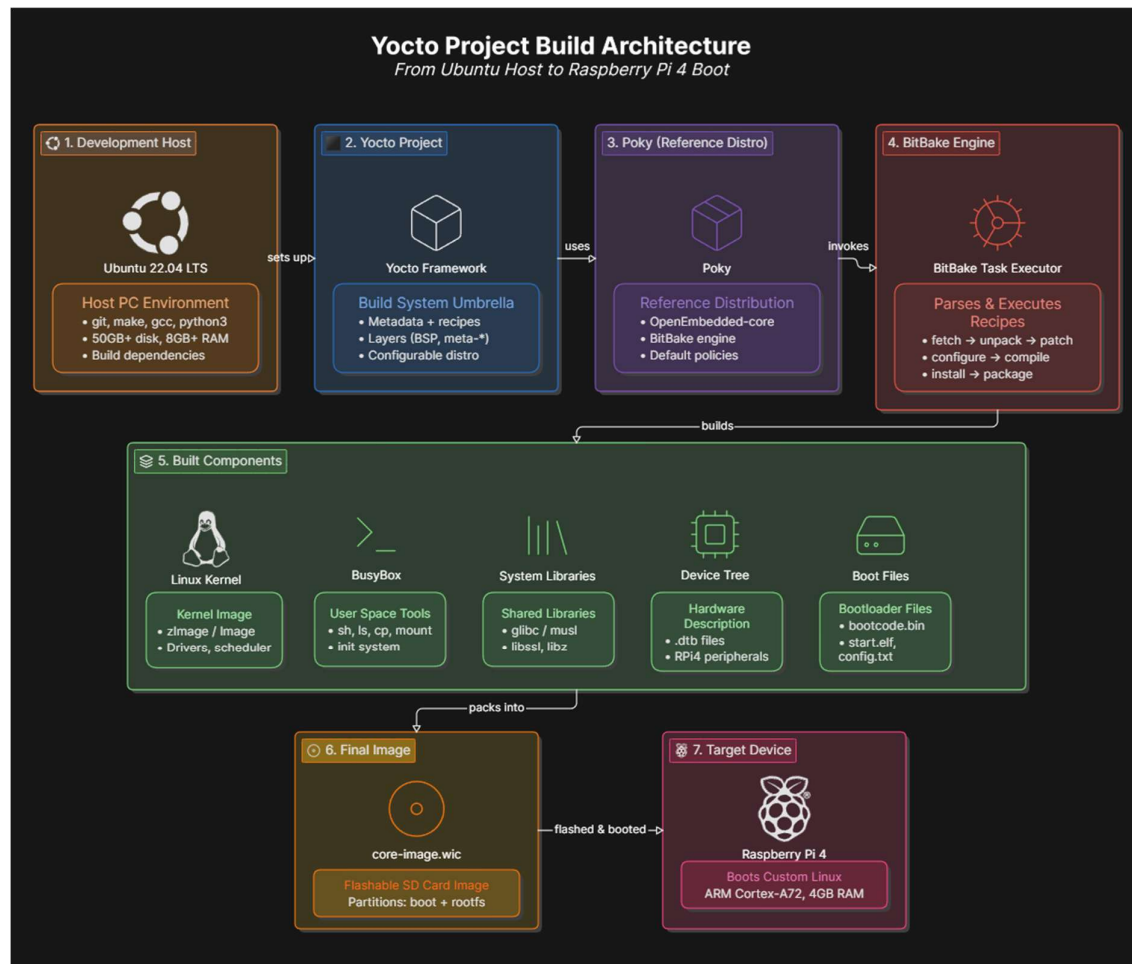
Custom Image

Advantages:

- ◆ Reproducible
- ◆ Small footprint
- ◆ Industrial standard
- ◆ Version controlled
- ◆ Cross-compilation support

Applications:

- ◆ Automotive OS
- ◆ Router OS
- ◆ Smart TV OS
- ◆ Medical Device OS

Project architecture:

Inputs to the project:

System requirements:

- ◆ Ubuntu OS
- ◆ Free 100gb of storage
- ◆ Target board (rasberry pi)
- ◆ SD card

* Important directories:

Before we get into the project directories we need to know about poky.

POKY:

What is poky?

Poky (pronounced *Pock-ee*) is the official **reference distribution**.

Poky is not a pre-compiled, ready-to-install Linux image like an Ubuntu .iso file. Instead, it is a combined Git repository containing the specific components needed to run a build.

When poky is cloned from git:

```
Yocto-project/
|_____poky/
```

What does poky contain?

```
poky/
|_____meta/
|_____meta-poky/
|_____meta-yocto-bsp/
|_____meta-raspberrypi/
|_____bitbake/
|_____scripts/
|_____oe-init-build-env
```

Explanation of each file or folder:

->**Meta**: It contains the foundational recipes, configurations, and classes required to build a functional, bootable Linux operating system.

recipes/: These folders contain the actual software blueprints (.bb and .bbappend files). They are grouped by function:

- recipes-core/: Essential boot components (systemd, sysvinit, glibc, busybox, base-files).
- recipes-kernel/: The Linux kernel and modules.
- recipes-connectivity/: Networking tools (wpa_supplicant, openssh, bluez).
- recipes-devtools/: Development utilities and compilers (gcc, python, git).
- recipes-extended/: Standard Linux utilities (tar, gzip, bash).

**** .bb files (recipe) contains where to download source, how to configure, how to compile, how to package**

Each recipe executes:

```
do_fetch      download source.
do_unpack     extract archives.
do_patch      applies board patches.
do_configure  runs ./configure and generates Makefiles and configurations.
do_compile    compilation.
do_install    copies files into staging area.
do_package    creates packages.
```

classes/: Contains .bbclass files. These are reusable code snippets that define common build behaviors. For example, cmake.bbclass tells BitBake how to build any software that uses CMake.

conf/: Holds core configuration files. The most important file here is `bitbake.conf`, which defines global variables, architecture details, and default directory paths used across the entire build system.

lib/: Contains Python modules and library scripts that extend BitBake's functionality and automate complex build steps.

site/: Contains site-specific caching configurations to help speed up the configuration and compilation steps for different hardware architectures.

Responsible for: 1. BusyBox 2. Init system 3. Base filesystem

->**Meta-poky/**: it is the configuration and policy layer (Distribution Layer) for Poky, the reference Embedded Linux distribution of the Yocto. It defines how the final operating system image behaves, outlining high-level distribution rules, configurations, and settings.

think of it as the corporate rulebook or the branding guidelines for your Linux operating system.

It contains: **poky.conf**

defines: package management, compiler options, build policies.

->**Meta-yocto-bsp/**: Meta-yocto-bsp is the Board Support Package (BSP) layer inside the reference Poky repository. It contains the essential hardware configuration files, kernel tweaks, and bootloader settings required to make the Yocto Project reference Linux images boot on specific target hardware.

->**Meta-raspberrypi/**: The official, community-maintained Board Support Package (BSP) layer for all Raspberry Pi hardware.

-If you want to build a Yocto Linux image that actually boots on a Raspberry Pi 3, 4, 5, or Zero, you swap out the reference meta-yocto-bsp layer and use meta-raspberrypi instead.

Contains:

Conf/machine/

Recipes-kernel/

Recipes-bsp/

Provides: raspberrypi4.conf raspberrypi4-64.conf

->**bitbake/**: BitBake is the engine and execution tool at the heart of the Yocto Project. It is not a Linux distribution itself, but a highly specialized, Python-based package management and build automation system.

Capable to do:

1. **Parses the metadata:** It reads all your configuration files (`.conf`), base recipes (`.bb`), and modifications (`.bbappend`) across all active layers (like meta-poky and meta-raspberrypi).
2. **Calculate dependencies:** It builds a massive dependency graph. If you want to install a text editor, BitBake figures out that it must first compile the C library, the compiler tools, and any required compression libraries.
3. **Execute task in parallel:** It schedules and runs tasks (like fetching code, unpacking, compiling, and packaging) across your computer's CPU cores simultaneously to speed up the build.

4. **Generates the file system:** It gathers all the compiled software packages and bundles them into a final bootable operating system image file (like an .iso or .wic file).

BUILD DIRECTORY STRUCTURE

This build directory is created after the environment is established.

We use

source oe-init-build-env

The command source oe-init-build-env is the initialization script that prepares your terminal for a Yocto Project build.

It does not install or compile anything. Instead, it sets up your environment variables and configures your workspace so that your terminal understands Yocto-specific commands like bitbake.

1. **Creates a Build Directory:** By default, it generates a new folder named build/ (if it does not already exist) and moves your terminal cursor inside it.
2. **Generates Configuration Files:** It populates the build/conf/ folder with starter files that you will need to customize your build:
 - I. local.conf: Where you set your target hardware (MACHINE) and download directories.
 - II. bblayers.conf: Where you list the active metadata layers (like meta-raspberrypi).
3. **Updates Environment Variables:** It adds the internal bitbake tools and script paths to your terminal's \$PATH variable. Without running this script, typing bitbake will result in a command not found error.

Directories created:

```
build/
|_____conf/
|_____downloads/
|_____sstate-cache/
|_____tmp/
```

CONF:

```
conf/
|_____local.conf
|_____bblayers.conf
```

Local.conf contains project configurations. (basically, a huge code that consists all the info about the build like cores involved, target variables, target machine etc)

Ex: MACHINE = "raspberrypi-4-64"

Bblayer contains meta, meta-poky, meta-yocto-bsp, meta-raspberrypi.

It tells bitbake where recipes are located.

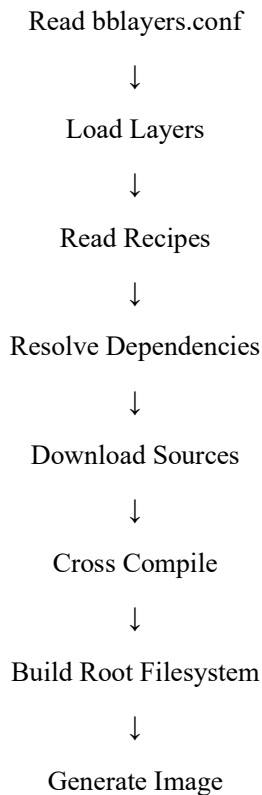
BITBAKE:

Internally, BitBake parses recipes to create a dependency graph of all required software packages. It then uses Python and Shell tasks to concurrently fetch, cross-compile, and package each component into isolated binary environments. Finally, it aggregates these packages to assemble a fully customized, bootable Linux root filesystem image.

Command:

```
bitbake core-image-minimal
```

internal working:

**DOWNLOAD FOLDER**

download/

contains: linux kernel source, busybox source, python source, openssl source.

SState Cache

Stores already build artifacts used to build the whole built again but not for a long time it just need 5 to 10 min to re build.

TMP directory

This is the largest directory which is primary workspace where BitBake performs all its processing, compilation, and assembly. It holds all temporary data, log files, middle-stage build artifacts, and your finalized operating system images.

```

tmp/
|_____work/
|_____deploy/
|_____sysroots/

```

Work: recipe build workspace.

Sysroots: target filesystem used during build. -contains headers, libraries, toolchain.

Deploy: the final output holder.

Generated output in:

Tmp/deploy/images/raspberrypi4-64/

Ex: manifest, spdx, testdata, wic.bz2.

Manifest: lists installed packages ex. Busybox, base-files, init-ifupdown.

SPDX: software bills ex. Licensing, security, compilation.

Testdata: metadata for testing.

.wic (image): contains everything needed to boot ex. Boot partitions, kernel, device tree, root filesystem.

The final image will be in the form of image.wic which is needed to be exported into an external storage i.e SD card.

Then the raspberry pi will be booted and the image gets flashed.

Generally the project just shows a terminal interface through which we can do simple testing with commands like

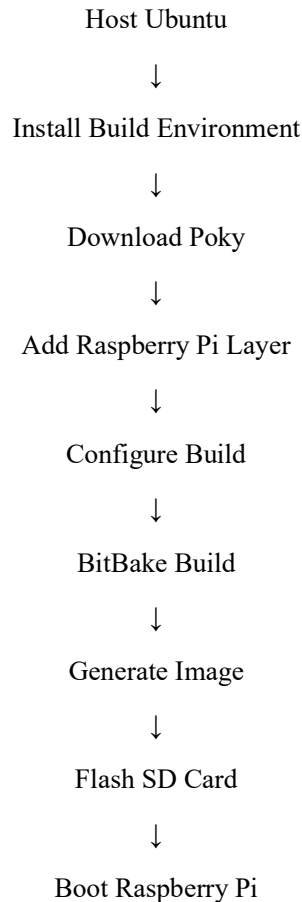
Ls

Uname -a

df -h

free -h

fp addr etc.....

End to End project flow with explanation of commands and their actions internally**Step 1: create project directory**

```
mkdir ~/yocto-project
```

```
cd ~/yocto-project
```

Here the linux creates a working directory for the project using mkdir, generally this is the workspace

Step 2: clone poky

```
git clone -b scarthgap git://git.yoctoproject.org/poky
```

This command downloads a specific version of the core Yocto Project development environment to your computer, once the command finished a folder named poky will be uploaded it contains everything needed to start the building the custom linux.

```
yocto-project/  
└─ poky/
```


The Git Server streams the compressed objects to the machine, which the local Git client validates and links to rebuild the project's timeline (Commit Reconstruction). Finally, Git unpacks these database records into your local directory as readable, editable files (Working Tree Creation).

poky/

|_____meta/

|_____meta-poky/

|_____meta-yocto-bsp/

|_____meta-raspberrypi/

|_____bitbake/

|_____scripts/

Folder	Purpose
meta	Core recipes
meta-poky	Distribution config
meta-yocto-bsp	Board configs
bitbake	Build engine

Step 3: add raspberry pi Support

```
git clone -b scarthgap https://github.com/agherzan/meta-raspberrypi
```

This command downloads the official Raspberry Pi Board Support Package (BSP) hardware layer configured for the Scarthgap version of the Yocto Project.

Internally creates:

poky/meta-raspberrypi

it adds support of raspberry pi 4, GPU, Kernal, Device tree, Boot config

download:

1. recipes-kernel
2. recipes-core
3. machine configs
4. device trees
5. boot files

Step 4: Initialize build environment

Cd poky

Source oe-init-build-env

Internally the script executes ow-init-build-env and creates a build folder where generally all the build will be saved

```

build/
|_____conf/
|           |_____local.conf
|           |_____bblayers.conf6
|_____downloads/
|_____sstate-cache/
|_____tmp/

```

Here the environment variables are loaded

Path bbpath buildir

And the build engine becomes active.

Step 5: Register layer

```
Bitbake-layer add-layer ../meta-raspberrypi
```

Internally modifies the .conf configuration

```
Build/conf/bblayer.conf
```

Before:

Meta

Meta-poky

Meta-yocto-bsp

After:

Meta

Meta-poky

Meta-yocto-bsp

Meta-raspberrypi -with this bitbake now knows the pi recipes exists.

Step 6: Configure target

```
nano conf/local.conf
```

this will open a huge code in which target specific variables exists which are needed to be changed according to the requirements, here we use raspberrypi as target so we need to change:

```
MACHINE ??= "raspberrypi4-64"
```

This internally changes the architecture and the compiler becomes aarch64-poky-linux-gcc.

Step 7: Start build**bitbake core-image-minimal**

this is the biggest, time taking yet important and main step in the project

phase 1: PARSE

reads all configuration, recipe, and include files (.conf, .bb, .bbappend, .inc) to discover every software package available to your system. BitBake evaluates these text files to dynamically calculate build variables, resolve cross-recipe dependencies, and map out the entire compilation sequence. The output is a massive, unified cache database containing the exact blueprint required to construct your specific Linux image.

Reads:

```
.bb
.bbappend
.conf
.inc
```

Ex: 961 recipes parsed (parsed – process of taking raw data and analysing it to make structures format)

phase 2: DEPENDENCY GRAPH

BitBake analyses the parsed recipes to construct a Directed Acyclic Graph (DAG), which acts as a strict flowchart of prerequisites for every software package. This graph ensures that fundamental foundations like glibc and the kernel are compiled and ready before dependent packages like busybox begin building.

Builds:

```
DAG
```

```
Ex. Busybox  ->  glibc  ->  kernel
```

Phase 3: FETCH

Downloads source and stores it in downloads

Ex. Busybox.tar.gx, linux.tar.gz

phase 4: CROSS COMPILE

the compiler “aarch64-poky-linux-gcc” cross compiles the files and generate the binary files such as

```
.o      .a      .so      etc.....
```

phase 5: PACKAGE

BitBake splits the compiled software into these package types, separating runtime files, development headers, and debugging symbols so they can be securely fed into the final root filesystem.

It creates rmp ipk deb etc...

phase 6: ROOT FILESYSTEM

creates a root file system for the project

```

rootfs/
|_____bin
|_____etc
|_____usr
|_____lib

```

phase 7: GENERATE BOOT IMAGE

creates the images i.e .img files

```

kernel          dtb          boot files

```

phase 8: GENERATE FINAL IMAGE

```
tmp/deploy/images/raspberrypi4-64/
```

```

generated:      core-image-minimal.rootfs.wic,bar2
                 manifest
                 spdx
                 testdata

```

Step 8: DECOMPRESS IMAGE

```
cp *.wic.bz2 image.wic.bz2
```

```
bunzip2 image.wic.bz2
```

STEP 9: FLASH SD CARD

```
sudo dd if=image.wic of=/dev/sdb bs=4M status=progress conv=fsync
```

internally:

```

read(image.wic)
  ↓
buffer
  ↓
write()
  ↓
block device

```

image written sector by sector.

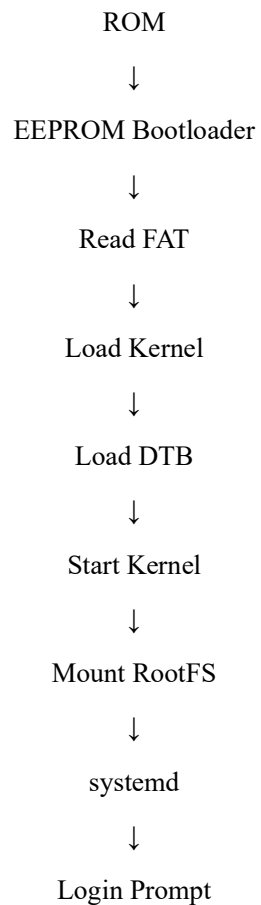
Step 11: SYNC**sync**

Command sync internally and flushes page cache, kernalbuffer, filesystem, cache. To SD card

Step 12: target boot

Insert the SC card and power on the board

Internal structure

**FINAL FILES GENERATED SYSTEM**

boot/

|___ kernel8.img

|___ bcm2711-rpi-4.dtb

rootfs/

|___ bin

|___ etc

|___ lib

Output of the project:

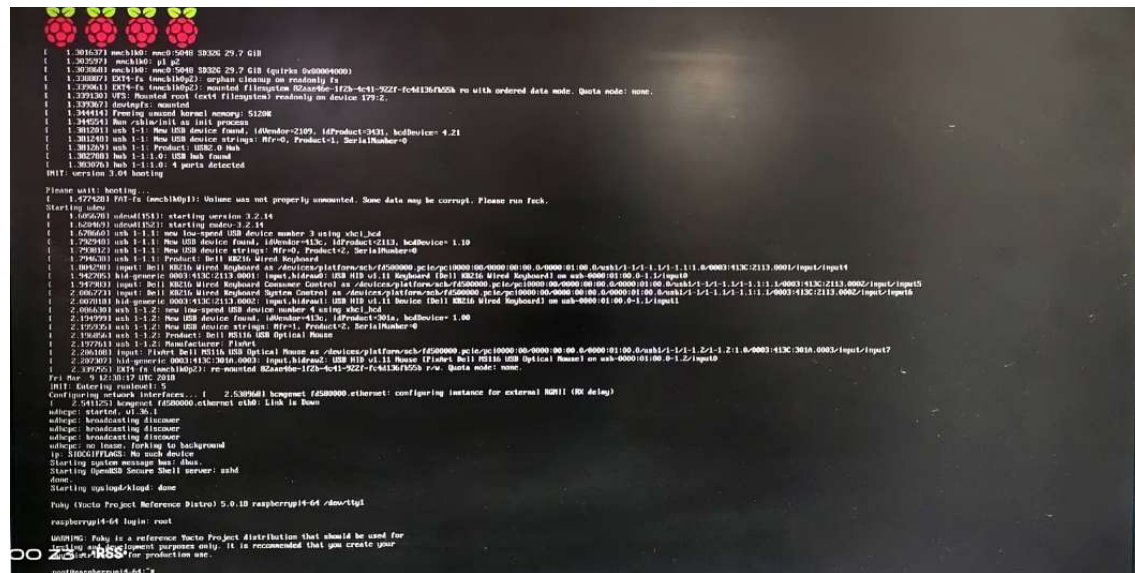


Fig 1: Booting stage - Custom linux os build by Yocto

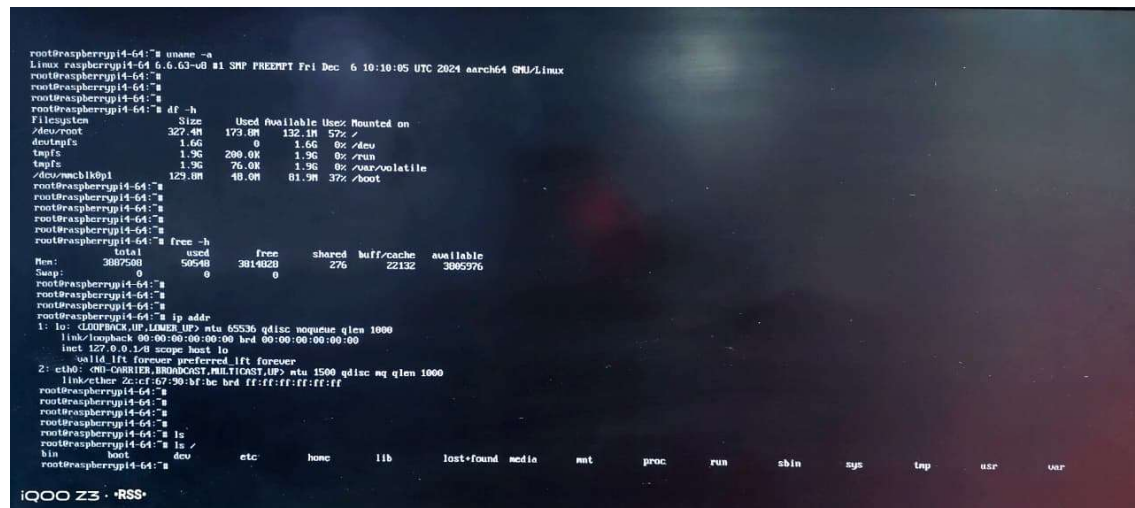


Fig 2: basic commands testing on in custom built linux OS using Yocto